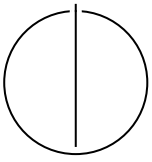
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Comparative Analysis of Widely Used Zipfian Distribution Implementations

Rodrigo Felix Forno



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

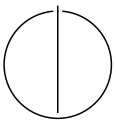
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Comparative Analysis of Widely Used Zipfian Distribution Implementations

Vergleichende Analyse von häufig verwendenten Implementierungen der Zipf-Verteilung

Author:	Rodrigo Felix Forno
Examiner:	Prof. Dr. Viktor Leis
Supervisor:	Bohyun Lee M.Sc., Gabriel Haas M.Sc.
Submission Date:	25th of March, 2025



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 25th of March, 2025 Rodrigo Felix Forno

Abstract

This thesis analyzes commonly used algorithms for sampling from the Zipfian distribution, with a focus on their application in benchmarking storage systems and database technologies. Since the late 1980s, there has been growing interest in evaluating system performance under skewed workloads—often modeled using the Zipfian distribution. While the theoretical foundations of Zipfian sampling are well established, practical implementations differ significantly in terms of accuracy and efficiency, which can lead to misleading benchmarking results.

This work provides a comprehensive comparison of Zipfian variate generators found in widely used benchmarking tools across academia and industry. It evaluates their precision, speed, and suitability for large-scale system testing. Two tools were developed to support this analysis: a benchmarking framework and a set of custom variate generators implemented in C++. Performance results show that Java-based implementations generate approximately 27 million samples per second, while their C/C++ counterparts achieve up to 44 million. In terms of accuracy, implementations using the Rejection-Inversion method produced frequency ratios close to 1 and a maximal absolute error of approximately 10^{-4}. Empirically, convergence was observed after generating roughly ten times the size of the distribution's support.

The evaluation reveals that most existing implementations are adequate for performance testing, though certain tools - such as FIO - exhibit significant inaccuracies and should be avoided. Additionally, while Rejection-Inversion remains the most accurate and efficient method, table lookup approaches demonstrate promising performance and may serve as viable alternatives with further refinement.

Contents

1 Introduction

The Zipfian distribution is a type of power-law distribution that appears in a wide range of natural and artificial phenomena, including the frequency of words in natural language, city populations, and website traffic patterns [1]. In recent years, it has also become highly relevant in the context of storage systems, databases, and network analysis, where operations such as read/write access or packet flows are often heavily skewed [2, 3]. In such scenarios, a small subset of elements is accessed disproportionately often, while the majority remain rarely used.

This skewed access pattern places specific demands on system design and performance testing. To emulate real-world workloads, benchmark tools must generate samples that follow the Zipfian distribution—efficiently, accurately, and at scale. This is especially important when the number of unique elements (i.e., the distribution's support size) is large. Several tools are commonly used for this purpose, including YCSB, FIO, and Sysbench [4]. However, the implementations of Zipfian variate generation in these tools vary, and have not been comprehensively compared in terms of performance or statistical accuracy. Deviations in accuracy could introduce biases significant enough to affect benchmark results [5].

This thesis addresses that gap. It provides a systematic review of Zipfian variate generation methods in widely-used benchmarking tools and investigates alternative sampling algorithms drawn from the broader literature on discrete random variate generation. The main contributions of this work are:

- A review and comparison of Zipfian distribution implementations in common benchmarking tools.
- The implementation of alternative variate generation methods, drawn from outside the storage and systems community.
- A performance and accuracy evaluation of all methods, with recommendations based on empirical results.

By examining both existing tools and novel approaches, this work aims to provide guidance for future benchmarking practices and contribute to more reliable and representative performance evaluations.

2 Background

The Zipfian distribution (also known as *Zipf's Law* [6]) is a power law; that is, the relationship between the probability of an event and the quantity that describes the event is given by an exponential factor [7]. A simple example of a power law is the relationship of the *area of a square*, $A(x)$, and its *side length*, x, given by $A(x) = x^2$. A formal definition is given below, parametrized by what we will define as the *skew* - α.

$$f(x) = C \cdot x^{\alpha} \tag{2.1}$$

Zipf's Law While a power-law, Zipf's is slightly different than the area of a square in that it is defined as a set of discrete values. For instance, when parsing a text and collecting the number of occurrences of each word, there are only finitely many words. The same occurs for databases: a key to a table has only a finite number of values. Moreover, the law says that the word rank - a word's position or key in terms of popularity - is inversely proportional to its frequency, given a skew factor. Formally, it is defined as follows:

$$\textbf{frequency} \propto \frac{1}{\textbf{rank}^{\alpha}} \tag{2.2}$$

An example is given in Figure 2.1. A text was parsed and a list of top five most popular words has been compiled: "the", "a", "an", "of", "to". The frequency is defined to be $freq(word) = \frac{occurences(word)}{\sum_{allwords} occurences(words)}$. We observe that the frequencies sum up to 1, and that the relationship between rank and frequency is not linear, but closely resembles an exponential curve.

The generalized Zipfian The previous example and intuition are helpful in understanding the pattern that this law presents, however, it is necessary to define it more deeply. From now on, the following conventions will be used:

- N - The support size of the distribution (e.g. *number of different words*)
- k - The rank of the *variate*[1] (e.g. *rank of word*)

[1] this is used interchangebly with *sample*

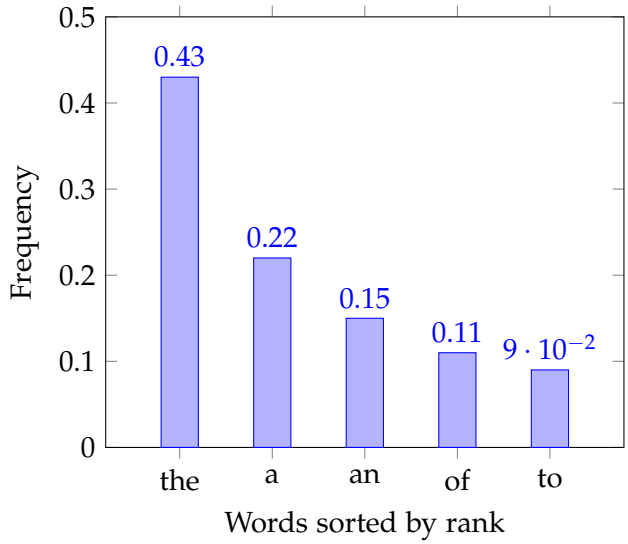


Figure 2.1: Example of a Zipfian distribution

Since the aim is generating variates according to this law, it is necessary to treat the *generalized Zipfian* as a statistical distribution. Below follows its Probability mass function (PMF):

$$f(k, N, \alpha) := \begin{cases} \frac{1}{H_{N,\alpha}} \frac{1}{k^\alpha} & 1 \leq k \leq N \\ 0 & k < 1 \text{ or } N < k \end{cases} \tag{2.3}$$

The term $H_{N,\alpha}$ is defined as the *generalized harmonic number* [7], acting as the normalizing constant C seen in 2.1 and ensuring the PMF sums to 1. It is defined as:

$$H_{N,\alpha} = \sum_{k=1}^{N} \frac{1}{k^\alpha} \tag{2.4}$$

As done before with Zipf's Law, the *Zipfian* distribution is plotted in Figure 2.2 as a line plot, this time with a *support size* of 100 distinct words. Now that all relevant terms are defined, we turn to the original problem of generating *samples*.

2.1 Discrete Variate Generation

The goal of *discrete variate generation* is to produce a set $\tilde{X}$ of m variates contained in the *support interval* $[0, N]$, such that $freq(x_i) \sim Dist(\alpha)$ for $i \in [0, m]$. In other words, the samples follow a given distribution $Dist(\alpha)$. To achieve this, all sampling algorithms use an random *uniform* distribution U, and transform these uniform *samples* into the

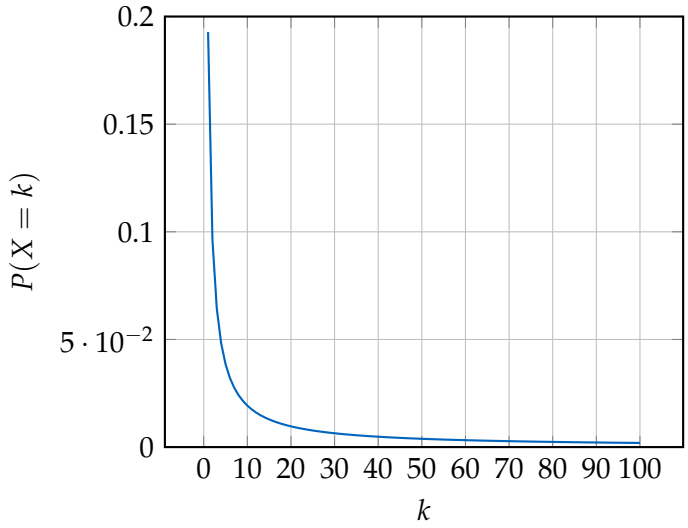


Figure 2.2: Probability Mass Function of a Zipfian distribution with $N = 100$ and $\alpha = 1$

target samples [8]. As distributions have different properties, some sampling methods are better than others for a particular distribution. Knowing this, the discussion is restricted to the algorithms listed below, as literature confirms these methods are better than alternatives (see [9, 10, 8]).

1. Rejection Sampling [9]
2. Rejection-Inversion Sampling [11]
3. Condensed Table Lookup [12]
4. Approximative sampling [2]

The first two methods, *Rejection Sampling* and *Rejection-Inversion Sampling*, are **exact**, whereas the last two, *Condensed Table Lookup* and *Approximative Sampling*, are **approximations**. Consequently, given enough samples, the first two should converge to a *Zipfian* distribution if implemented correctly, whereas the latter two might have systematic deviations [13]. The following subsections will explain each algorithm.

2.1.1 Rejection Sampling

Rejection sampling works by *rejecting* samples from a *uniform* distribution using a *bounding function* to decide which samples are kept and which are thrown away. A good example is a dartboard, onto which darts are thrown at random. If the target distribution is normal, we draw its contour (the *bounding function*) in the dartboard and *accept* the darts that fall below it and *reject* the ones above. The graphs in Figure 2.3 illustrate this principle well - blue points are accepted darts, whereas red points are the rejected ones.

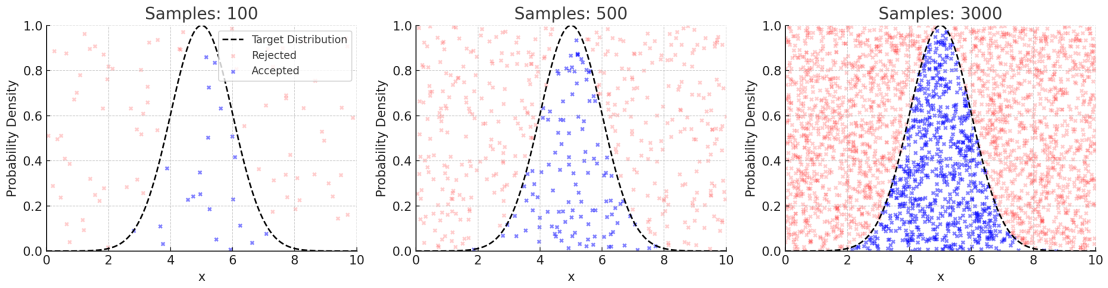


Figure 2.3: To generate these graphs, a random uniform sample is taken for the x-axis as well as another normalized one for the y-axis.

Similar to the normal distribution, a *bounding function* is chosen for the *Zipfian*: $1/k^{-\alpha}$. This function is easier to determine than the PMF in Equation 2.3, as we skip the computationally expensive normalization constant. The only requirement on this function is that it should *be greater up to a constant factor* than the original PMF. A disadvantage of this method is that two random samples are required: one for the x-axis, another for the y-axis.

2.1.2 Rejection-Inversion Sampling

Due to Hörmann and Derflinger [11], this method is a more efficient version of the Rejection Sampling method. It is based on the following observations regarding the naive rejection method:

- A naive rejection algorithm requires two samples to generate the target variate.
- The Zipfian is bounded by a monotonically decreasing and convex function, which makes it possible to invert the bounding function.

These two observations lead to the following principle:

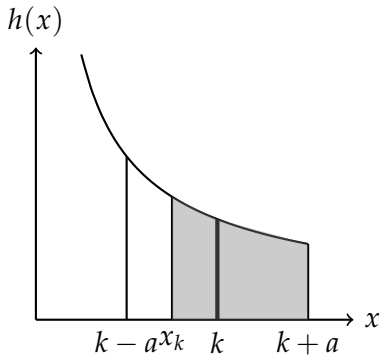


Figure 2.4: Rejection-Inversion Sampling for a Zipfian distribution

> Generate a random variate u from the uniform distribution $U(0,1)$ and invert the bounding function to find the corresponding $x = H^{-1}(u)$. Define a bounding area around the nearest integer k (with radius a). Accept X as a sample if $x \geq x_k$ and $x \leq k + a$. Otherwise, reject it.[2]

To visualize the principle used, consider Figure 2.4: After k was found using the inversion method, the bounding area is defined for radius a. Therefore an interval beginning at $k - a$ and ending at $k + a$ is defined, then used to accept or reject x. Typically, a is chosen to be $1/2$ [11].

2.1.3 Condensed Table Lookup

An alternative to exact methods is computing probabilities of a function according to a PMF for all *ranks* in the domain and storing them in a table. Since this probability is defined in an interval $[0, 1]$, it is possible to take a random *uniform* sample from the interval $[0, 1]$ and look up the entry stored at the matching probability. This sampling technique is called a *table lookup* method [12]. Consider Figure 2.5: A sample from an *uniform* distribution (the implementation is a Pseudo-Random Number Generator (PRNG)) is taken and used to find the corresponding index in the table.

One limitation of this approach is the size of the array being in $O(n)$, which is not feasible for large *support* N. To circumvent this, Marsaglia and Tsang proposed a *condensed table lookup method* [12], which works based on the principle exposed in the table below.

[2]This is a simplified version of the algorithm. For a more detailed explanation, see [11]

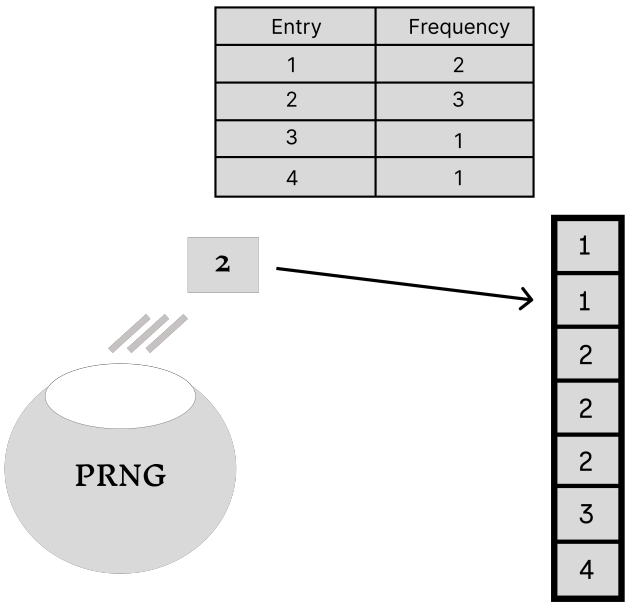


Figure 2.5: The table contains the frequency of each index, whereas the array is populated with as many entries as there are occurrences of each index. The sample "2" was generated, which is used to access the second position in the array

A floating-point number p with d decimal places can be used to represent a probability. For example, assuming $d = 2$, the entry one from the last figure has probability $p(1) = 0.28$. To realize this probability, a table of size 10^2 is required. In order to achieve reasonable precision, a table of size in the magnitude of 10^{10} would be required, which would be unfeasable. Alternatively, less space can be used by breaking this table down to d smaller tables. To do this, each probability gets split into d digits m, which respectively get m entries in the corresponding table. Now, generating a uniform integer from the interval $[1, 100]$, it is possible to access the correct table and generate the variate. [3]

[3]This is a simplified version of the algorithm. For a more detailed explanation, see [12]

2.1.4 Approximation Sampling

Due to Gray [2], this method is a fast approximation of the *Zipfian's* tail and the generalized harmonic number. The first few probabilities are calculated precisely and normalized by the approximated $H_{N,\alpha}$, while an exponential curve approximates the tail. A uniform sample is then used to generate a *Zipfian* variate using inversion. Figure 2.6 makes this idea clear by having two different areas along the x-axis.

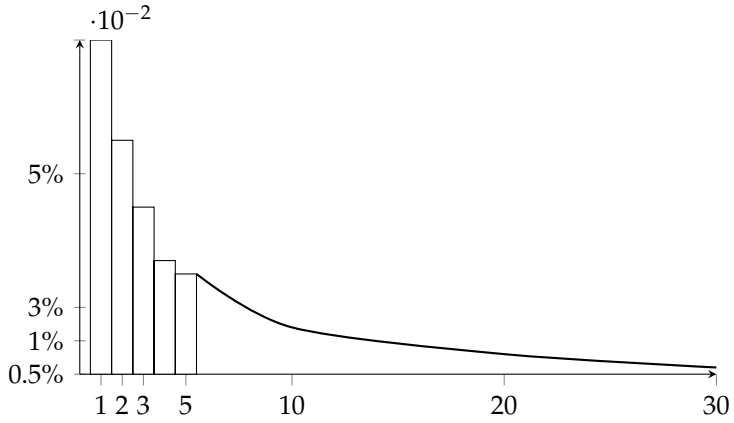


Figure 2.6: Jim Gray's method for generating samples from a Zipfian distribution

The mathematical description of this method is not quite straightforward and is omitted here. For a more detailed explanation, see [2, 14].

Summary As all generation techniques were described, the question remains how to evaluate them for accuracy compared with the theoretical probability values yielded by the PMF in Equation 2.3 - even for *exact* methods. As these methods are developed, some assumptions [13] are taken that might not hold up in the computers used to generate these variates:

- There exists a perfectly uniform generator that produces a sequence of *independent* random variables uniformly distributed in $(0, 1)$
- Computers can save and manipulate real numbers

Both of these assumptions cannot be fulfilled. First, the *number generators* used are only pseudo-random [8] and might have biases and produce correlated subsequences. For the second, computers only have limited precision, and computing terms such

as the *generalized harmonic number* suffer from imprecisions caused by *roundups* and *cutoffs* [13]. Some distributions such as *power laws* worsen this issue [15]. Therefore, we introduce a diverse set of criteria in the following sections for identifying correct implementations and measuring their errors:

- Maximal absolute cumulative density function (CDF) difference
- Sum of absolute PMF differences

2.2 Error Metrics

To compare the different methods introduced previously, it is imperative to first define the framework used to evaluate them. Some principles can be applied from the literature [16, 13, 17]:

1. Calculate the theoretical distribution for the given parameters: N - support size and α - skew.
2. Generate an empirical distribution using the algorithms described earlier.
3. Use a **goodness-of-fit test** to compare the generated distribution to the theoretical one.

This framework assumes a "black-box" view of the generating processes and treats the generated samples as a dataset that should be fitted to the theoretical Zipfian. At the center of this comparison is a **goodness-of-fit test**, which tells whether the empirical distribution matches the expected distribution closely enough. As such, the test requires a *comparison metric* between the two distributions - a function that takes two distributions and returns a number that quantifies their dissimilarity. Let us look at the example given in Figure 2.7. If the blue curve represents the theoretical distribution and the orange curve is the empirical one, the shaded area represents the difference between the two. The smaller this area, the better the empirical distribution fits the theoretical one.

For this work, two measures are going to be used: the **maximal absolute CDF difference**, known in the literature as the *Kolmorgorov-Smirnov* statistic and the **sum of absolute PMF differences**, known in the literature as the *Total Variation distance* [18, 13]. The former is widely used for hypothesis testing and has the advantage of being a non-parametric test, flexible enough for a wide range of distributions [19]. It is however sensitive to outliers. The latter is a more general measure of the difference between two probability distributions and, on the other hand, is more resistant against outliers.

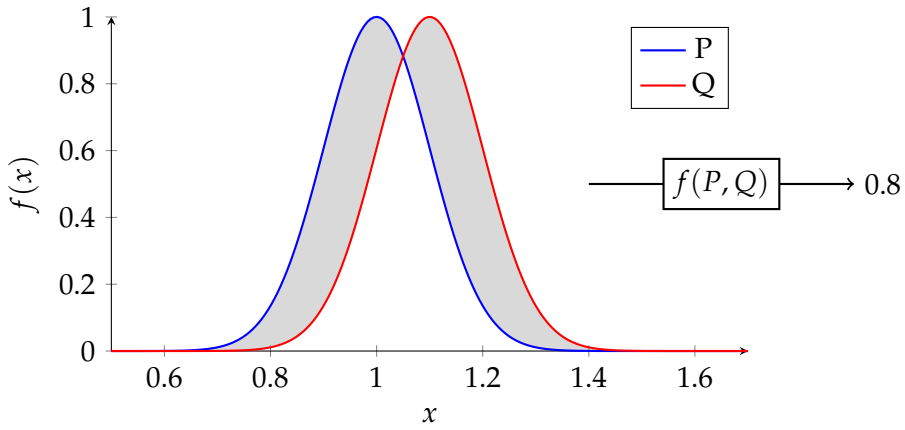


Figure 2.7: Accuracy Measure

Maximal absolute CDF difference This measure is defined as the maximum difference - $D_n(P,Q)$ - between the empirical and theoretical CDF. Equation 2.5 formalizes it. It has a nice graphical interpretation as the maximal vertical distance between the two CDFs, as seen in Figure 2.8.

$$D_n = \sup_x |P(x) - Q(x)| \tag{2.5}$$

This measure is then compared to a *standard value* given by a probability function called the *Kolmogorov distribution* [18].

$$F_n = P[D_n \leq x | \mathcal{H}_0] \quad \text{for } x \in [0,1] \tag{2.6}$$

Due to its computational complexity, an exact form of this distribution is prohibitive for bigger *support sizes* N. Consequently, approximations are prefered, such as the one given by Marsaglia [18]. The implementation used in the *SciPy* library follows this work.

Sum of absolute PMF differences This distance, also known as *Total Variation Distance (TVD)*, is given by the sum of all absolute differences in the probability mass function of two distributions. More precisely, given two (discrete) distributions, P and Q, defined in region X:

$$d_{TV}(P,Q) = \frac{1}{2} \sum_{x \in X} |P(x) - Q(x)| \tag{2.7}$$

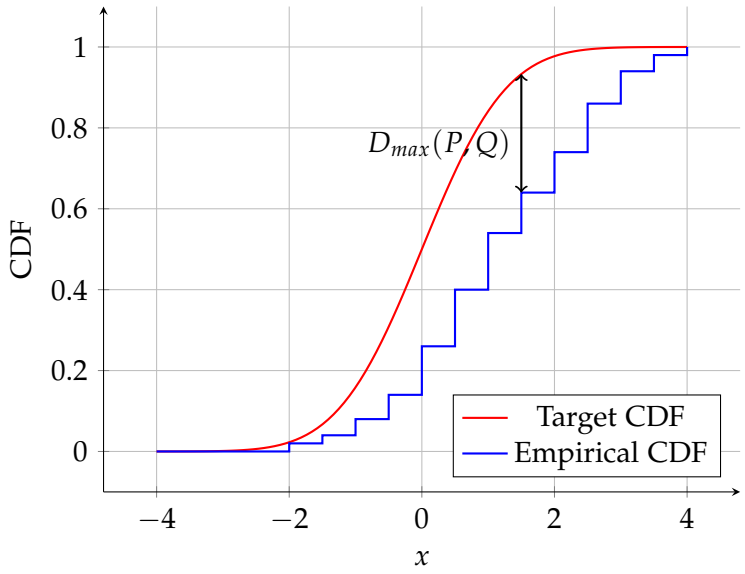


Figure 2.8: Kolmogorov-Smirnov Test

The distance ranges from 0 to 1, where zero means the distributions are equal, and 1 means they are completely different. Interpreting it as a percentage allows statements such as: "About y% of the probability mass differs between the theoretical and empirical distribution". Figure 2.7 was first presented as a general measure, but is also, in fact, the *TVD*. An important property of the total variation distance is that with growing sample sizes, the distance between the empirical and theoretical distribution should converge to 0 for all x [20].

As all required notions and metrics are well defined, a detailed description of the research approach comes next. With it, common benchmarking tools will be identified in the literature and their popularity evaluated.

3 Approach

This thesis aims to identify the most widely used frameworks for generating synthetic *Zipfian* distributed data in the systems programming and database community and make a comparative evaluation. The first step required to achieve this is a systematic review of current approaches. The following sections describe

1. The **research approach** used to identify the frameworks
2. The **common frameworks** recognized from the search along with short descriptions
3. The **popularity and impact** of those tools
4. The **benchmarking module** which will be used to evaluate the frameworks

3.1 Research Approach

First, a list of relevant conferences and journals was defined to identify frameworks the systems programming and database community used. The six following conferences were taken into account:

- Proceedings of the VLDB Endowment (PVLDB)
- ACM Special Interest Group on Management of Data (SIGMOD)
- Conference on Innovative Data Systems Research (CIDR)
- USENIX Symposium on Operating Systems Design and Implementation (OSDI)
- USENIX Symposium on Networked Systems Design and Implementation (NSDI)
- USENIX Conference on File and Storage Technologies (FAST)

These conferences were chosen due to their high selectivity and special relevancy for storage technology research, where standardized IO benchmarks are prevalent . The search was conducted broadly following the steps indicated in [21], as defined below:

1. Word search on keyword search matrix (see table 3.1)
2. Identify the frameworks used in the papers
3. Extended keyword search, this time adding the frameworks identified in the previous step

The first step suggests the use of a *keyword matrix*. Such a matrix can be built by choosing appropriate keywords that refer to the topic of interest, so all words should be present in the works searched. Synonyms are used to improve the accuracy and scope of the search. It is defined as follows:

	Keyword 1	Keyword 2	Keyword 3
synonyms	zipf zipfian	benchmark workload evaluation empirical dataset	synthetic generated sampled sampler

Table 3.1: Keyword search matrix

Given this matrix, the search was conducted on the following academic databases: *IEEE Xplore*, *ACM Digital Library*, *DBLP*, and *Google Scholar*. The results were sorted in *relevance* order, such that well-cited but also relatively new papers appear at the top. An analysis of the findings follows in the next section.

3.2 Common Frameworks

In the first phase, the following frameworks were identified among the top 20 results of each conference

Frameworks
Yahoo! Cloud Serving Benchmark (YCSB) [22]
Flexible I/O Tester (FIO) [23]
Sysbench [24]
OLTP-Bench [25]

The previous keyword matrix was extended with the frameworks' names to assert their popularity in the six venues. Pie charts in Figure 3.1 show the trends found.

The most popular framework is **YCSB**, followed by **OLTPBench**, as well as **FIO**. Although this would point to a clear winner in **YCSB**, it is important to note that the

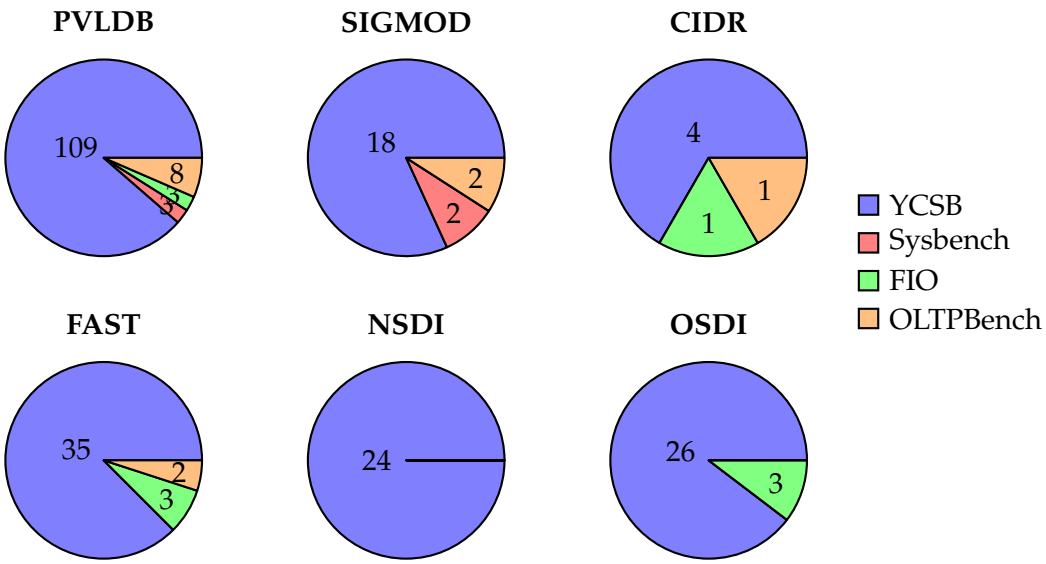


Figure 3.1: Popularity of the frameworks in the conferences

popularity of a framework considered in academic journals might not reflect its actual usage in practice. A search for the frameworks in *GitHub* yielded the table 3.1 (as of 22 March 2025), containing the number of *starts* given to a code repository.

Framework	Stars
YCSB	5k
Sysbench	6.3k
FIO	5.5k
OLTPBench	0.5k

Table 3.2: Popularity of the frameworks on GitHub

3.2.1 Yahoo! Cloud Serving Benchmark

YCSB is a widely used benchmarking tool for cloud databases, developed by *Yahoo! Research* and now maintained by a community of developers. The framework generates *workloads* used to test the performance of database systems under stress, by populating tables with entries. How these entries are distributed is not fixed and can be chosen by

the user. One of the possible distributions is the *Zipfian*, and a submodule in YCSB is responsible for generating variates.

In order to compute the samples, the benchmark suite uses the algorithm proposed by Gray, as presented before (see 2.1.4). One limiting factor in his original paper is that generated keys are lexicographically sorted, e.g. key 0 has the highest frequency. The keys should be randomly distributed within the support distribution to better reflect real workloads. See Figure 3.2 for a comparison of these two cases.

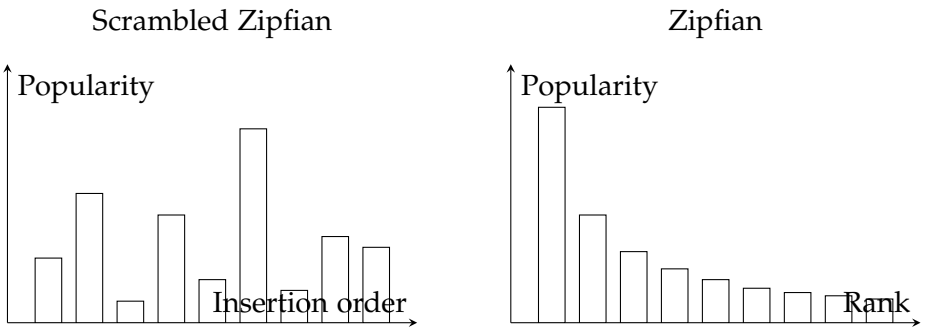


Figure 3.2: Comparison of Jim Gray's version (right) and the one employed by YCSB (left).

To avoid the issue described earlier, YCSB uses a hashing function to randomly spread the values in support of the distribution and employs an extended domain to reduce collisions [22].

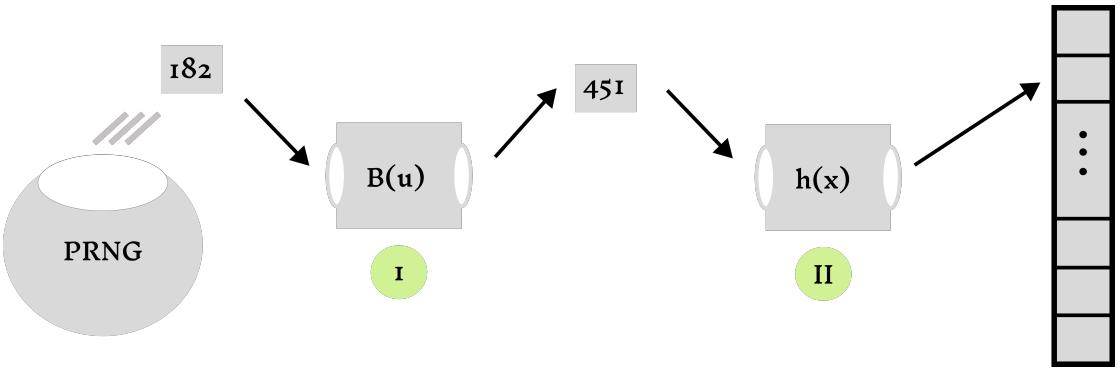


Figure 3.3: YCSB does sampling by inversion (1) and then hashes the value to spread in in the available range (2)

3.2.2 Flexible I/O Tester (FIO)

FIO [23] is a benchmarking tool for storage systems. Much like YCSB, its purpose is generating workloads following a certain *access pattern* to test a storage system. The workloads can vary in type, including sequential or random, and read-only, write-only, or mixed. In the case of **random I/O**, the user can change the workload to simulate both asymmetrical and more symmetrical workloads. The underlying distributions include the *Zipfian*, *Normal*, *Pareto*, and *Uniform* distributions.

The algorithm used is also the approximative method due to Jim Gray. As with YCSB, the same problem arises with the lexographical ordering of the results, to which *FIO* resorts to using a hashing function called *lookup3*, in the *Jenkins* collection of hash functions [26].

3.2.3 Sysbench

Sysbench is a benchmarking tool for databases as well as storage systems. It works very similarly to YCSB, using multi-threading to strain the system with concurrent accesses. The tool can generate synthetic data according to different probability distributions, among which the Zipfian distribution.

What sets it apart from the previous is that the variate generation uses a different algorithm. The one employed by Sysbench is based on *Rejection-Inversion sampling* (see Subsection 2.1.2), and the implementation closely follows that from the Apache Commons Math library [27] under *RejectionInversionZipfSampler*.

3.2.4 OLTP-Bench

The OLTP-Bench [25] strives to unify different benchmarks for evaluating a Database Management System (DBMS). It favors a modular design, enabling very different benchmarks to be used under a standard interface, which makes it extensible. However, it also has enough benchmarks built-in to be used directly, including AuctionMark [28], Transaction Processing Performance Council Benchmark C (TPC-C), and YCSB. The focus will be on the AuctionMark benchmark, as it is the one excepting YCSB, which generates synthetic data following a Zipfian distribution. The implementation is almost directly based on YCSB, thus not further differentiated.

3.3 Benchmarking Module

A benchmarking module was developed to evaluate the frameworks. It was written in Python and allows the user to specify which sample generator to use, as well as the

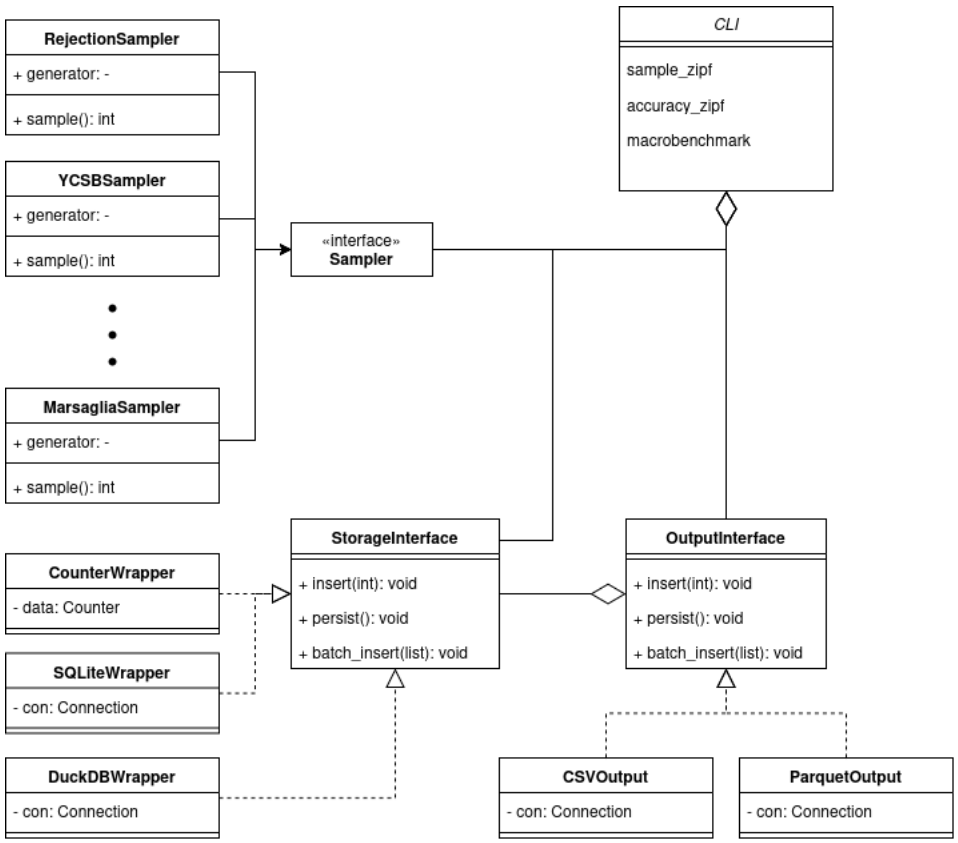


Figure 3.4: UML diagram of the benchmarking module

distribution parameters. It is usable from the command line and includes the following commands: `sample_zipf`, to generate a Zipfian distribution, `accuracy_zipf` to calculate accuracy measures for one distribution, and `benchmark` to run a specified performance benchmark. It follows a modular design, with the following interfaces: *SamplerInterface*, responsible for providing a common wrapper interface to all the sampling algorithms; *StorageInterface*, which defines how the data is to be stored in memory before processing; and *OutputInterface*, which defines how the data is to be persisted. A UML diagram of the tool is shown in Figure 3.4.

To use the libraries and frameworks described previously, only the necessary code functionality for generating *Zipfian* variates was extracted and packaged as *shared libraries* and/or *executables*. These were compiled with optimization turned on to ensure a fair comparison.

3.3.1 Commands

To generate a sample, the command `sample_zipf` is used. For example,

```
sample_zipf --generator sysbench --n 1000 --samples 1000 --skew 1.5 --
    output csv --storage sqlite --buckets 100
```

Running the command will create a CSV file with 1000 samples following a Zipfian distribution with a skew of 1.5 and a domain from 0 to 1000. The samples will be stored in memory in a SQLite database with 100 buckets.

In order to calculate the accuracy measures of the distribution implied by a sample stored in a CSV, the command `accuracy_zipf` is used, as in

```
accuracy_zipf --input_dir /path/to/csv --skew 1.5 --output_dir /path/to/
    output
```

This method assumes that the directory contains CSV files of results of various sample sizes and domain sizes, belonging to one generator. It will then generate 3 CSV files, with columns for domain size and rows for domain sizes - The first one with the Total Variation Distance (TVD), the second with the Kolmogorov-Smirnov (KS) distance, and the third with the p-value of the KS test.

4 Evaluation

In the previous chapters, four common frameworks and their underlying algorithms were presented. To make an accurate and representative comparison, it is necessary to evaluate the accuracy of the distributions generated, as well as the performance of the algorithms used to generate them.

- **Accuracy**: The statistical properties of the generated distributions will be compared to the expected values given by the theoretical Zipfian distribution
- **Performance**: The time taken to generate the distributions will be measured and compared using standard profiling tools as well as custom-written benchmarks

The variate generators considered were presented in the previous chapters: YCSB, FIO, Sysbench, and OLTP-Bench (which will not be further separated from YCSB), along with implementations of the algorithms discussed in Chapter 2: Rejection and Condensed Table Lookup. Additionally, the *Apache Commons* [27] implementation of the Rejection-inversion algorithm will be evaluated to compare it to its C counterpart. To make comparison easier, the following *aliases* will be used:

Framework/Algorithm	**Alias**
YCSB	Approximative (Java)
Sysbench	Rejection-Inversion (C)
FIO	Approximative (C)
OLTPBench	Approximative (Java)
Rejection	Rejection (C++)
Condensed Table Lookup	Condensed Table Lookup (C++)
Apache Commons	Rejection-Inversion (Java)

Table 4.1: Aliased used in Evaluation

4.1 Experimental Setup

This section describes the hardware and software environment used to conduct the experiments. All experiments described in this chapter were conducted on a machine

with the specifications detailed in table 4.2. The software environment used can be found in Table 4.3, and compiler versions in Table 4.4.

CPU Model	Clockspeed	Number of Cores	Memory Capacity	Memory Speed
AMD Ryzen 7 5850U	3.6 GHz	8	16 GB	3200 MHz

Table 4.2: Computer Specifications

Operating System	Kernel Version	Relevant Tools
Debian GNU/Linux 12	6.1.0-amd64	perf[29], JMH [30], Google Benchmark [31]

Table 4.3: Software Specification

Language/Version	Compiler	Benchmark/Sampler
C++20	gcc 12.2.0	Own implementation
Java 17	OpenJDK 17.0.8	YCSB, OLTP-Bench
C (GNU17)	gcc 12.2.0	Sysbench, FIO

Table 4.4: Environment Specification

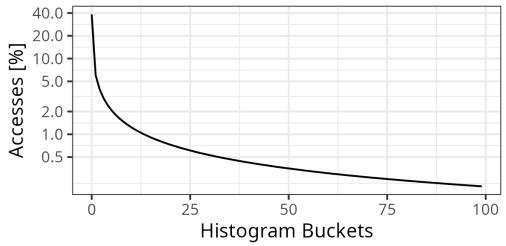
4.2 Methodology

Three quantifiable criteria will be used to measure **accuracy** to offer a well-rounded view of the quality of the generated distributions - the **maximal absolute CDF difference** and the **sum of absolute PMF differences** as explained previously in Chapter 2. They are well explored in the literature [16, 13, 32, 17]. Moreover, **frequency ratio graphs** will be used to evaluate accuracy through the whole *support* of the distribution, allowing to observe biases related to the *rank*.

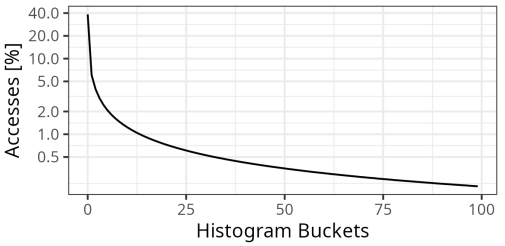
The main measure of interest for **performance** is *throughput* - the number of variates generated in a time interval (e.g. 10 million/ms). However, measuring this quantity is not trivial, as benchmarking in modern systems is unreliable if done inappropriately [30, 33]. Some precautions are taken to avoid this, which will be explained later in Section 4.4.

4.3 Evaluating accuracy

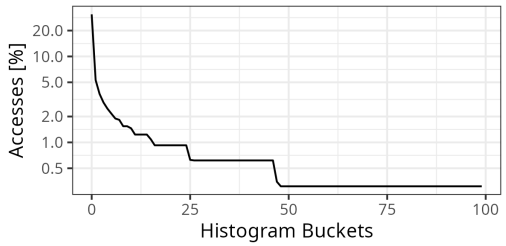
Probability graphs The first step is to visualize the distributions each generator produces, which allows us to identify patterns and inconsistencies in the generated samples.



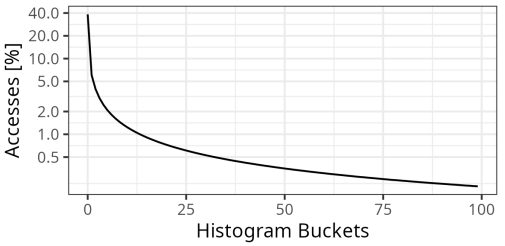
(a) Rejection-Inversion (Java)



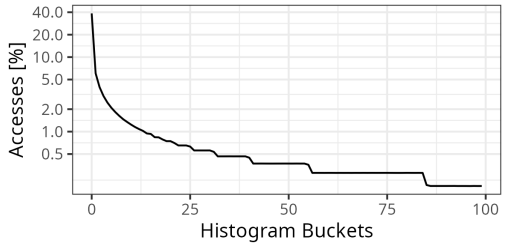
(b) Approximative (Java)



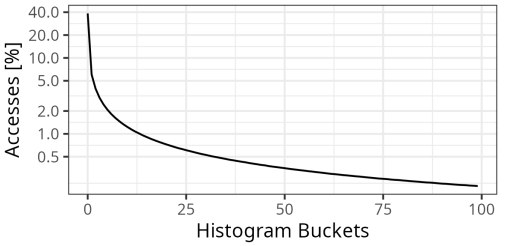
(c) Approximative (C)



(d) Rejection-Inversion (Java)



(e) Condensed Table Lookup (C++)



(f) Rejection (C++)

Figure 4.1: PMF of all implementations with parameters: $N = 100\text{M}$, $\alpha = 0.8$, samples = 1B

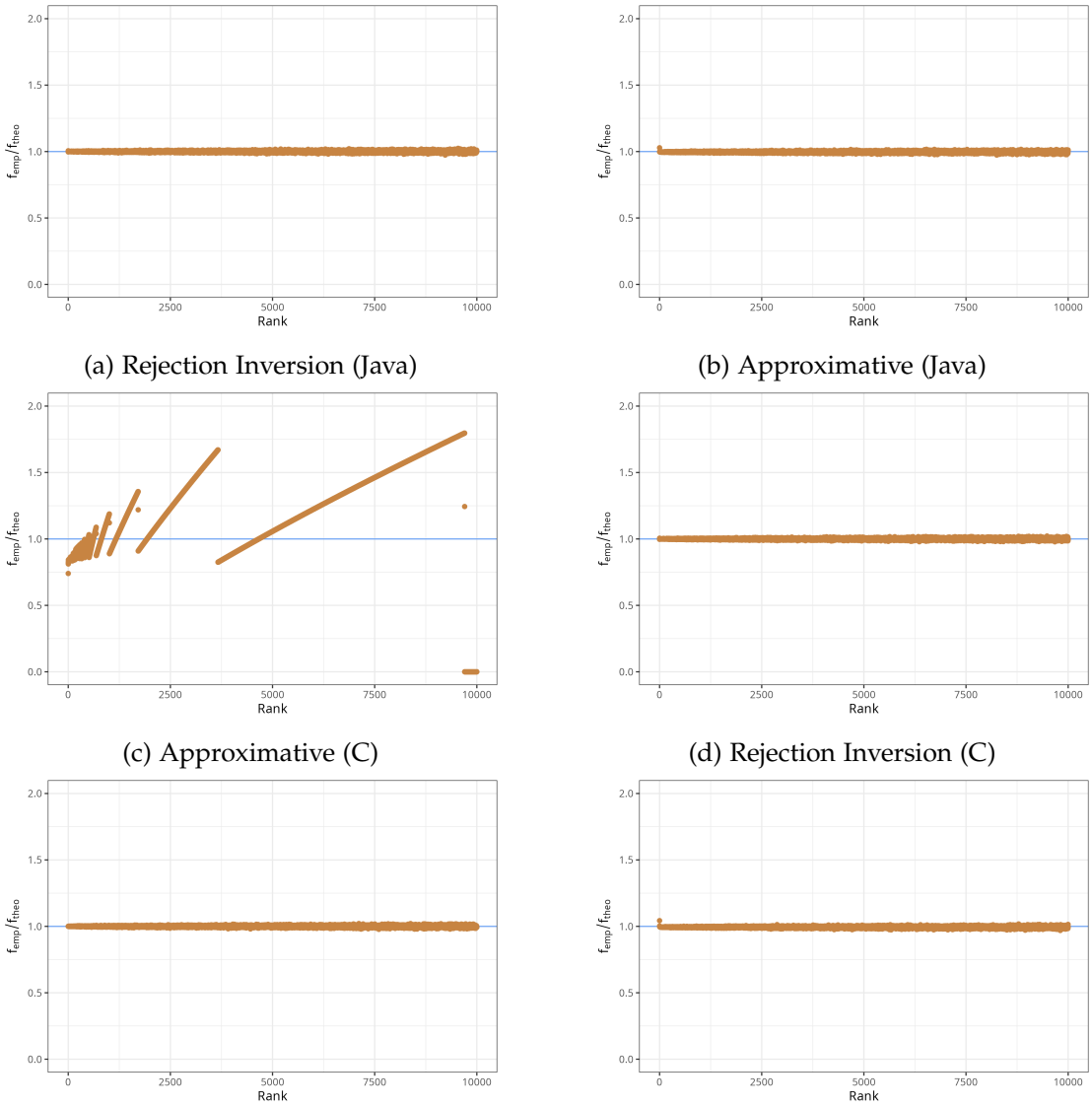
There is a noticeable "staircase" effect happening with *Approximative (C)* and *Condensed Table Lookup (C++)*, though the first is more acute due to the non-linearity of the *y-axis*. The implementation were evaluated to investigate possible causes.

The *Approximative (C)* approach uses the same algorithm as its counterpart in Java. What differs between both is the choice of hashing, in both cases posterior to the

generation of the variate according to the algorithm exposed by Gray [2]. This points to the choice of hasing function of the *C* version, but a rigorous analysis requires a twofold approach: First, disable hashing and evaluate the resulting distribution. Secondly, exchange the hashing algorithm used, which is 32-bit based (investigating the source code reveal it is from a *lookup3* from the *Jenkin's* family of hashing functions [26]) for a 64-bit version. While this approach goes in a promissing direction, it was not possible to verify it in this work and is left open.

As for the case of the condensed table lookup, the problems lies in precision limitations of the generated tables - it is limited to 2^{30}, or rather $5 \cdot 10^9$. A cutoff implies the algorithm cannot compute $f_{theo}(k)$ that differs more than a factor of 10^{-9}, such that many *ranks* will be "stored" in the same index. Consequently, stretches of the *ranks* in the x-axis will have the same probability, resulting in the lines seen. Such a mechanism explains the "staircase" effect seen in Figure 4.1. Morever, the absolute difference between consecutive $f_{theo}(k)$ decreases as k increases, as can be observed by the "plateau" lines getting longer. Currently, an alternative that avoids the "staircase" effect is not known, and further investigation was not possible.

Frequency ratio graphs To further analyse how the accuracy varies with changing *rank* of the *empirical/generated* distributions, the **frequency ratio graph** was used. It consists of the *rank k* in the *x-axis* and the ratio $\frac{f_{emp}(k)}{f_{theo}(k)}$ in the *y-axis*. The graphs in Figure 4.2 show the results for the six variate generators. We notice some patterns: the implementations in graph B and F overestimate the first rank, such that $f_{emp}(k) > f_{theo}(k)$; and graph C presents an angled striped pattern. A clear cause for the artifact seen in graph B and F could not be verified and is a direction of improvement for this thesis. The second pattern can be explained by same phenomenon found in the *PMF* graph. As the rank k and $f_{emp}(k)$ stays constant for long intervals, $f_{theo}(k)$ increases. Eventually $f_{emp}(k)$ changes and the process begins again.



(a) Rejection Inversion (Java)

(b) Approximative (Java)

(c) Approximative (C)

(d) Rejection Inversion (C)

(e) Condensed Table Lookup (C++)

(f) Rejection (C++)

Figure 4.2: All distributions have support size 10^6 and were sampled 10^9 times

Maximal absolute CDF difference While the previous analysis helped identify and explain big divergences in the behavior of the generators, a more careful comparison should still be conducted. Using metrics and statistical test, it is possible to determine whether an algorithm is accurate up to 90% certainty. The first such metric is $D_n = \sup_x |P(x) - Q()|$, and it was calculated for each implementation, as seen in Table 4.5.

We observe that the largest error of the CDF of the Approximative (Java) implementation is about fourty times bigger than the equivalent for the *Rejection-Inversion* algorithm. This is expected, as the latter is an exact method. On the other hand, the *Approximative (Java)* algorithm and the *Condensed Table Lookup* have high error readings, which is expected given that they are *approximative*. To account for the precise error readings is, however, beyond the scope of this work and requires a sofisticated mathematical analysis.

Sampler	**Maximal CDF distance**	**P-value**
Approximative (C)	0.08664	0
Approximative (Java)	0.00086	0
Rejection-Inversion (Java)	0.00002	0.96998
Rejection-Inversion (C)	0.00002	0.96998
Condensed Table Lookup (C++)	0.00471	0
Rejection (C++)	0.00139	0

Table 4.5: KS distance metric for 10^9 samples

From the table, we can conclude that only *Rejection-Inversion* is adequate enough. But as this distance is sensitive to outliers [17], the *Approximative (Java)* algorithm is not yet discarded.

Sum of probability differences To offer a more holistic view of the possible biases in the variate generators, a second metric is introduced - $d_{TV}(P, Q) = \frac{1}{2}\sum_{x \in X} |P(x) - Q(x)|$. As the literature does not directly suggest its usage as a test statistic [34], an arbitrary limit $\epsilon = 0.01$ is chosen: up to 1% of the probability mass of the empirical distribution F_{emp} can deviate from the F_{theo}. Table 4.6 summarizes the results, which can be read as "The Rejection algorithm has 3% difference in probability mass compared to the theoretical value". Using the ϵ criterium defined, we accept *Approximative (Java)* and *Rejection-Inversion (both)*.

Discussion of accuracy results Considering both metrics, the *Rejection-inversion* and *Approximative (Java)* implementations will be accepted. The Condensed table lookup does not pass the test, but could be further investigated, as table lookups are quite fast [12]. It might seem to be the case that these requirements were very strict, since a 10^{-3} difference in the CDF might seen small. However, small deviations could cause significant increases in latency and an example of the latency in memory accesses can help illustrate this point. Drawing from relevant literature [35, 36], we have that *L2*

Sampler	Sum of PMF differences
Approximative (C)	0.2010
Rejection (C++)	0.0311
Condensed Table Lookup (C++)	0.0164
Approximative (Java)	0.0101
Rejection-Inversion (Java)	0.0096
Rejection-Inversion (C)	0.0096

Table 4.6: TVD metric results for 10^9 samples

Cache has a latency of around 4 ns, whereas *DRAM* can be approximate at around 100 ns. Assuming that 95% of accesses hit the cache and then 5% go to the main memory, it is possible to calculate the expected latency for both the true F_{theo} and a F_{emp} with $d_{TV} = 0.1$. Let L_{true} be the former and L_{wrong} the latter:

$$\tilde{L}_{true} = \mathbb{E}(L_{true}) = 0.95 \cdot 4ns + 0.05 \cdot 100ns = 8.8ns \tag{4.1}$$

$$\tilde{L}_{wrong} = \mathbb{E}(L_{wrong}) = 0.85 \cdot 4ns + 0.15 \cdot 100ns = 18.4ns \tag{4.2}$$

This would create a relative error of $\frac{18.4-8.8}{8.8} = 109\%$, which is quite significant. An equivalent calculation for the $\epsilon = 0.01$ used in the analysis yields a relative error of only 10%, which is much more acceptable.

4.4 Evaluating Performance

Measuring performance presents difficulties in modern hardware, as controlling for external factors outside of the segment of code being tested is nontrivial. To better isolate the *code to test*, some techniques are applied [30, 33, 37], as exposed below:

- Set specific CPU core for the process generating samples
- Disable frequency scaling
- Disable turbo boost
- Fix the maximal amount of available memory
- Warm-up runs

The languages of interest are C++, C, and Java; therefore, the benchmarking tools are Google Benchmark [31] and JMH [30]. For the latter, some further precautions are required:

- Use the `Blackhole` object to consume the results of the computation
- Use the `CompilerControl` annotation to avoid inlining
- Use the `State` annotation to avoid dead code elimination

At last, it is important to define the runs. Since what is going to be measured is throughput, we define the duration of a single run to be 1 second. This will be repeated thirty times and the average throughput will be calculated. Since the speed is invariant to *support size* N and *skew* α, these are kept at 10^8 and 0.8, respectively.

For Google Benchmark, the following command will be used to run the benchmarks (for more details on how to use the tool, see [31]):

```
./benchmark --benchmark_repetitions=30 --benchmark_min_warmup_time=1.0
```

As a precaution for the sample generation not be optimized away, the function `benchmark::DoNotOptimize` was used on the resulting variable. The same procedure was reproduced for the JMH (more details on [30, 38]), yielding the following command:

```
java -jar target/benchmarks.jar -bm thrpt -wi 10 -i 30 -f 1 -r 1
```

Sampler	**Average Throughput (M/s)**	**Standard Deviation (M/s)**
Approximative (C)	26.2	0.01
Approximative (Java)	29.9	0.14
Rejection-Inversion (Java)	27.4	0.07
Rejection-Inversion (C)	43.6	0.15
Condensed Table Lookup (C++)	45.2	0.06
Rejection (C++)	28.1	0.30

Table 4.7: Microbenchmarking Results

From Table 4.7, we can recognize that the Condensed Table Lookup sampler is the fastest. The reason for it is that the lookup is extremely fast and the costly part is offloaded to initialization. The Sysbench sampler is the second fastest, which is expected given the overhead of the JVM of its Java counterpart. Nonetheless, the difference between Java and C is explored, for which using a profiler is necessary. Running `perf` [29] along with the JMH tool yields the following results for YCSB and Rejection-Inversion (Java). Only relevant lines are presented.

Rejection-Inversion (Java)

```
Baseline:  565.140.532.065 instructions 4,33 insn per cycle
Sampler:  204.065.670.512 instructions 1,56 insn per cycle
Sampler:  11.310.090 L1-dcache-load-misses 0,02% of all L1-dcache
accesses
```

YCSB

```
Baseline:  578.884.719.478 instructions 4,43 insn per cycle
Sampler :  392.403.334.414 instructions 2,92 insn per cycle
Sampler:  339.375.866 L1-dcache-load-misses 0,43 % of all L1-dcache
accesses
Sampler:  789.273 L1-icache-load-misses 0,30% of all L1-icache
accesses
```

Given that both the baseline and the actual sample routine have similarly high IPC and low number of data and instruction cache misses, the lower throughput in Java is likely not caused by an inefficient implementatation, but rather by the overhead inherent to the JVM runtime. These aspects will not be explored further, which limits a rigorous conclusion. Regardless, this points towards the JVM being responsible for the difference in performance.

5 Future Work

This thesis provided a comprehensive overview of existing methods for sampling from a Zipfian distribution in the context of database and storage systems research. However, further exploration of the broader field of power-law variate generation could uncover additional sampling algorithms that exploit structural properties specific to the Zipfian distribution [6].

Another promising avenue for future research lies in table-based methods. Historically, these were often dismissed due to memory limitations [11]. Yet, this thesis has demonstrated their viability in modern environments, despite falling short of delivering a competitive implementation.

Extending the evaluation framework to include other commonly used distributions - such as normal or Pareto - could also prove valuable, as these access patterns are frequently employed in performance benchmarking [22].

Additionally, this work identified a gap in the literature concerning the use of error metrics for evaluating the accuracy of non-uniform variate generators. While it is well-known that digital computation introduces inaccuracies in such processes [15, 13, 17], there is little consensus on which metrics are most appropriate. As a result, the metrics chosen in this thesis prioritized interpretability, but future research could benefit from a more rigorous framework for error evaluation.

6 Conclusion

This thesis investigated the state-of-the-art in discrete variate generation for Zipfian distributions within benchmarking applications used in storage and database systems research. Four widely-used benchmarking tools were identified and analyzed: YCSB, FIO, Sysbench, and OLTP-Bench, each employing its own implementation of Zipfian variate generation.

In parallel, a survey of relevant literature in variate generation led to the identification of four promising methods: Rejection-Inversion Sampling, Rejection Sampling, Condensed Table Lookup, and Approximative Sampling. Of these, Rejection Sampling and Condensed Table Lookup were implemented, as the others are already in use within the studied tools.

The evaluation, which measured both performance and statistical accuracy, revealed that Sysbench and the Condensed Table Lookup method (both C-based) demonstrated strong performance. However, the latter lacked sufficient accuracy in the evaluated metrics, warranting further investigation before practical adoption. FIO was found to be both inaccurate and among the slowest implementations, making it unsuitable in its current form. Similarly, the Rejection Sampling method performed poorly in terms of both speed and accuracy.

Based on these findings, this work recommends using YCSB and Sysbench for benchmarking tasks requiring Zipfian distributions. Additionally, updating FIO with an improved hashing function could make it a viable option as well.

Abbreviations

PVLDB Proceedings of the VLDB Endowment

SIGMOD ACM Special Interest Group on Management of Data

CIDR Conference on Innovative Data Systems Research

OSDI USENIX Symposium on Operating Systems Design and Implementation

NSDI USENIX Symposium on Networked Systems Design and Implementation

FAST USENIX Conference on File and Storage Technologies

YCSB Yahoo! Cloud Serving Benchmark

FIO Flexible I/O Tester

DBMS Database Management System

TPC-C Transaction Processing Performance Council Benchmark C

PRNG Pseudo-Random Number Generator

TVD Total Variation Distance

KS Kolmogorov-Smirnov

PMF Probability mass function

CDF cumulative density function

List of Figures

List of Tables

Bibliography

[1] B. B. Mandelbrot, "Is Nature Fractal?" *Science*, vol. 279, no. 5352, pp. 783–783, Feb. 6, 1998. DOI: 10.1126/science.279.5352.783c.

[2] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger, "Quickly generating billion-record synthetic databases," *SIGMOD Rec.*, vol. 23, no. 2, pp. 243–252, May 24, 1994, ISSN: 0163-5808. DOI: 10.1145/191843.191886.

[3] M. E. Crovella, "Performance Evaluation with Heavy Tailed Distributions," in *Computer Performance Evaluation.Modelling Techniques and Tools*, B. R. Haverkort, H. C. Bohnenkamp, and C. U. Smith, Eds., Berlin, Heidelberg: Springer, 2000, pp. 1–9, ISBN: 978-3-540-46429-7. DOI: 10.1007/3-540-46429-8_1.

[4] "Navigating the Database Performance Landscape: A Comprehensive Guide to Database Benchmarking Suites," benchANT. (2024), [Online]. Available: https://benchant.com/blog/benchmarking-suites.

[5] "The Tail at Scale." (), [Online]. Available: https://research.google/pubs/the-tail-at-scale/ (visited on 03/10/2025).

[6] D. M. W. Powers, "Applications and explanations of Zipf's law," in *Proceedings of the Joint Conferences on New Methods in Language Processing and Computational Natural Language Learning - NeMLaP3/CoNLL '98*, Sydney, Australia: Association for Computational Linguistics, 1998, p. 151, ISBN: 978-0-7258-0634-7. DOI: 10.3115/1603899.1603924.

[7] M. E. J. Newman, "Power laws, Pareto distributions and Zipf's law," *Contemporary Physics*, vol. 46, no. 5, pp. 323–351, Sep. 2005, ISSN: 0010-7514, 1366-5812. DOI: 10.1080/00107510500052444. arXiv: cond-mat/0412004.

[8] M. E. Kuhl, "History of random variate generation," in *Proceedings of the 2017 Winter Simulation Conference*, ser. WSC '17, Las Vegas, Nevada: IEEE Press, Dec. 3, 2017, pp. 1–12, ISBN: 978-1-5386-3427-1.

[9] L. Devroye, "Discrete Univariate Distributions," in *Non-Uniform Random Variate Generation*, L. Devroye, Ed., New York, NY: Springer, 1986, pp. 485–553, ISBN: 978-1-4613-8643-8. DOI: 10.1007/978-1-4613-8643-8_10.

[10] R. A. Kronmal and A. V. Peterson, "On the Alias Method for Generating Random Variables from a Discrete Distribution," *The American Statistician*, vol. 33, no. 4, pp. 214–218, Nov. 1979, ISSN: 0003-1305, 1537-2731. DOI: 10.1080/00031305.1979.10482697.

[11] W. Hörmann and G. Derflinger, "Rejection-inversion to generate variates from monotone discrete distributions," *ACM Transactions on Modeling and Computer Simulation*, vol. 6, no. 3, pp. 169–184, Jul. 1996, ISSN: 1049-3301, 1558-1195. DOI: 10.1145/235025.235029.

[12] G. Marsaglia, W. W. Tsang, and J. Wang, "Fast Generation of Discrete Random Variables," *Journal of Statistical Software*, vol. 11, no. 3, 2004, ISSN: 1548-7660. DOI: 10.18637/jss.v011.i03.

[13] J. Leydold, "Error Detection in Non-Uniform Random Variate Generators,"

[14] D. E. Knuth, *The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms*. USA: Addison-Wesley Longman Publishing Co., Inc., Oct. 1997, 784 pp., ISBN: 978-0-201-89684-8.

[15] F. Radicchi, "Underestimating extreme events in power-law behavior due to machine-dependent cutoffs," *Physical Review E*, vol. 90, no. 5, p. 050 801, Nov. 3, 2014, ISSN: 1539-3755, 1550-2376. DOI: 10.1103/PhysRevE.90.050801. arXiv: 1405.0058 [physics].

[16] A. Clauset, C. R. Shalizi, and M. E. J. Newman, "Power-law distributions in empirical data," *SIAM Review*, vol. 51, no. 4, pp. 661–703, Nov. 4, 2009, ISSN: 0036-1445, 1095-7200. DOI: 10.1137/070710111. arXiv: 0706.1062 [physics].

[17] J. F. Monahan, "Accuracy in random number generation," *Mathematics of Computation*, vol. 45, no. 172, pp. 559–568, 1985, ISSN: 0025-5718, 1088-6842. DOI: 10.1090/S0025-5718-1985-0804945-X.

[18] G. Marsaglia, W. W. Tsang, and J. Wang, "Evaluating Kolmogorov's Distribution," *Journal of Statistical Software*, vol. 8, pp. 1–4, Nov. 10, 2003, ISSN: 1548-7660. DOI: 10.18637/jss.v008.i18.

[19] "Kolmogorov–Smirnov Test," in *The Concise Encyclopedia of Statistics*, New York, NY: Springer, 2008, pp. 283–287, ISBN: 978-0-387-32833-1. DOI: 10.1007/978-0-387-32833-1_214.

[20] B. Keng. "The Empirical Distribution Function," Bounded Rationality. (Mar. 12, 2016), [Online]. Available: http://bjlkeng.github.io/posts/the-empirical-distribution-function/ (visited on 03/14/2025).

[21] W. M. Bramer, G. B. de Jonge, M. L. Rethlefsen, F. Mast, and J. Kleijnen, "A systematic approach to searching: An efficient and complete method to develop literature searches," *Journal of the Medical Library Association : JMLA*, vol. 106, no. 4, pp. 531–541, Oct. 2018, ISSN: 1536-5050. DOI: 10.5195/jmla.2018.283. PMID: 30271302.

[22] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the 1st ACM Symposium on Cloud Computing*, ser. SoCC '10, New York, NY, USA: Association for Computing Machinery, Jun. 10, 2010, pp. 143–154, ISBN: 978-1-4503-0036-0. DOI: 10.1145/1807128.1807152.

[23] "1. fio - Flexible I/O tester rev. 3.38 — fio 3.38-19-gd1eb documentation." (), [Online]. Available: https://fio.readthedocs.io/en/latest/fio_doc.html (visited on 01/16/2025).

[24] A. Kopytov, *Akopytov/sysbench*, Jan. 16, 2025.

[25] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux, "OLTP-Bench: An extensible testbed for benchmarking relational databases," *Proc. VLDB Endow.*, vol. 7, no. 4, pp. 277–288, Dec. 1, 2013, ISSN: 2150-8097. DOI: 10.14778/2732240.2732246.

[26] B. Jenkins. "Lookup3." (), [Online]. Available: http://burtleburtle.net/bob/c/lookup3.c (visited on 03/19/2025).

[27] "Math – Commons Math: The Apache Commons Mathematics Library." (), [Online]. Available: https://commons.apache.org/proper/commons-math/ (visited on 02/22/2025).

[28] "AuctionMark: An OLTP Benchmark for Shared-Nothing Database Management Systems « H-Store." (), [Online]. Available: https://hstore.cs.brown.edu/projects/auctionmark/ (visited on 02/22/2025).

[29] "Perf: Linux profiling with performance counters." (), [Online]. Available: https://perfwiki.github.io/main/ (visited on 03/09/2025).

[30] "Avoiding Benchmarking Pitfalls on the JVM." (), [Online]. Available: https://www.oracle.com/technical-resources/articles/java/architect-benchmarking.html (visited on 03/09/2025).

[31] "Google/benchmark: A microbenchmark support library." (), [Online]. Available: https://github.com/google/benchmark (visited on 03/09/2025).

[32] L. Deng and R. Chhikara, "Robustness of some non-uniform random variate generators," *Statistica Neerlandica*, vol. 46, no. 2–3, pp. 195–207, 1992, ISSN: 1467-9574. DOI: 10.1111/j.1467-9574.1992.tb01337.x.

[33] D. Beyer, S. Löwe, and P. Wendler, "Benchmarking and Resource Measurement," in *Model Checking Software*, B. Fischer and J. Geldenhuys, Eds., Cham: Springer International Publishing, 2015, pp. 160–178, ISBN: 978-3-319-23404-5. DOI: 10.1007/978-3-319-23404-5_12.

[34] E. L. Lehmann and J. P. Romano, "Large-Sample Optimality," in *Testing Statistical Hypotheses*, E. Lehmann and J. P. Romano, Eds., Cham: Springer International Publishing, 2022, pp. 709–772, ISBN: 978-3-030-70578-7. DOI: 10.1007/978-3-030-70578-7_15.

[35] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design, Fifth Edition: The Hardware/Software Interface*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., Sep. 2013, 800 pp., ISBN: 978-0-12-407726-3.

[36] "Teach Yourself Programming in Ten Years." (), [Online]. Available: http://norvig.com/21-days.html#answers (visited on 03/25/2025).

[37] "Benchmarking tips — LLVM 21.0.0git documentation." (), [Online]. Available: https://llvm.org/docs/Benchmarking.html (visited on 03/11/2025).

[38] J. Jenkov. "JMH - Java Microbenchmark Harness." (), [Online]. Available: http://jenkov.com/tutorials/java-performance/jmh.html (visited on 03/12/2025).